

Received 17 March 2023, accepted 31 March 2023, date of publication 5 April 2023, date of current version 14 April 2023.

Digital Object Identifier 10.1109/ACCESS.2023.3264584

RESEARCH ARTICLE

K-FL: Kalman Filter-Based Clustering Federated Learning Method

HYUNGBIN KIM¹, BYUNGCHAN KIM², YONGHO KIM², CHEOLWOO YOU², (Member, IEEE),
AND HYUNHEE PARK², (Member, IEEE)

¹Department of Computer Science and Engineering, Korea University, Seoul 02841, Republic of Korea

²Department of Information and Communication Engineering, Myongji University, Yongin 17058, Republic of Korea

Corresponding author: Hyunhee Park (hhpark@mju.ac.kr)

This work was supported in part by the National Research Foundation of Korea (NRF) and Institute for Information & Communications Technology Planning & Evaluation (IITP) funded by the Korean Government [Ministry of Science and ICT, South Korea (MSIT)], in part by the Acquisition Indication-Media Access Control (AI-MAC) Protocol on Distributed Machine Learning for Intelligent Flying Base Station under Grant 2022R1A2C2005705, and in part by the Development of the 6G Service Targeted Artificial Intelligence (AI)/Machine Learning (ML)-Based Autonomous Regulating Medium Access Control (6G STAR-MAC) under Grant 2021-0-00368.

ABSTRACT Federated learning is a distributed machine learning framework that enables a large number of devices to cooperatively train a model without data sharing. However, because federated learning trains a model using non-independent and identically distributed (non-IID) data stored at local devices, the weight divergence causes a performance loss. This paper focuses on solving the non-IID problems and proposes Kalman filter-based clustering federated learning method called K-FL to get performance gain by providing a specific model with low variance to the device. To the best of our knowledge, it is the first clustering federated learning method that can train a model requiring fewer communication rounds under the premise that non-IID environment without any prior knowledge and an initial value set by the user. From simulations, we demonstrate that the proposed K-FL can train a model much faster, requiring fewer communication rounds than FedAvg and LG-FedAvg when testing neural networks using the MNIST, FMNIST, and CIFAR-10 datasets. As a numerical result, it is shown that the accuracy is improved in all datasets while the computational time cost is reduced by $1.43\times$, $1.67\times$, and $1.63\times$ compared to FedAvg, respectively.

INDEX TERMS Federated learning, distributed machine learning, clustering method, kalman filter.

I. INTRODUCTION

With the significant development in hardware devices and software technologies, artificial intelligence (AI) has recently developed considerably and is being employed in various fields, such as user-customized recommendation systems, image recognition, and natural language processing [1]. AI involves a learning process to make human-like judgments. Here, a decision algorithm is generated by repeatedly learning various data; however, to train an AI decision algorithm, collecting massive amounts of data fit for the purpose is necessary. When transmitting the raw data of each device to the parameter server and during training of the decision algorithm, privacy issues often occur [2], [3].

The associate editor coordinating the review of this manuscript and approving it for publication was Asad Waqar Malik¹.

To address this data privacy issue, Google has proposed federated learning (FL) [4]. FL comprises numerous wireless devices, such as smartphones and a server, and has an architecture wherein decision algorithm training is conducted in each device rather than on the server [5]. FL can be divided into cross-silo and cross-device learning according to the environment involved, such as the number of devices (i.e., organizations) participating in the server [6]. The weights generated by a device after training are sent to the server, and the server aggregates these received weights and updates the global model; therefore, training is performed without transmitting the raw data used by each device to the server. Thus, FL has the advantage of ensuring data anonymity, which is different from existing machine learning methods because the raw data are not transmitted to the server during training. However, if the number of devices participating in FL increases, system heterogeneity, statistical heterogeneity

problems, and high communication costs may result owing to the non-independent and identically distributed (non-IID) data of each device [6]. In addition, owing to the non-IID of the number and shape of data of distributed devices, FL encounters a limitation in that it cannot realize cloud computing [6], [7], [8], [9], [10].

Thus, several studies have attempted to address this issue in FL. The authors of [7] directly limit the size of local updates by adding an L2 regularization term to the local objective function. FedProx improves the local objective function based on FedAvg [7]. FedProx limits the distance between the local and global models, ensuring that the global model does not deviate significantly from the global optimum value. In addition, they introduce a hyperparameter to control the weight of L2 regularization. Depending on the value set by a user, the approach may demonstrate high accuracy. The authors of [8] assume that each device has a different local epoch as it passes through communication rounds. Accordingly, devices with large local epochs are assumed to have a greater impact when they are aggregated into one global model on the server. FedNova improves the aggregation stage on the server to overcome the limitations of non-IID environments [8]. To prevent bias, FedNova performs normalization according to the local epoch for each device before sending the local model to the server. However, both methods may have few positive effects depending on the value set by the user and have the disadvantage of generating additional computational energy.

In [11], each device has some common data through the concept of a globally shared small subset of data. The authors in [11] show the trade-off between the test accuracy and the size of globally shared data. However, it is difficult to comprise shared data and have each device have the corresponding data in a real environment. The study in [7] adds a regularization term to the local optimization to reduce weight divergence between local and global models. This approach trains a single model and thereby improves the performance of the global model; however, one single model cannot work optimally for all devices in the non-IID environment of the local dataset [12]. The authors in [13] propose hierarchical clustering of local updates to improve training on non-IID data. The study in [13] is similar to that of [14], which proposed a clustered federated learning (CFL) procedure. [13] focuses on the computer vision image classification problem, and clusters the devices by the similarity of their local updates to reduce the number of communication rounds required to reach convergence in non-IID environments. However, it is necessary to set in advance the time point at which clustering is performed through the clustering step parameter n defined in [13]. Our approach does not require any prior knowledge and an initial value set by the user about the devices as only the local updates are used to form the cluster automatically.

In this study, we apply a Kalman filter-based clustering method on the server to improve the performance of the global model. However, before applying a clustering method,

it is important to identify the point at which clustering is to be performed. Thus, we apply a Kalman filter to set this clustering time using the prediction accuracy generated in the previous round. We propose a method of clustering by automatically finding a clustering point without prior knowledge or initial values by applying a Kalman filter. The proposed method predicts the prediction accuracy of the next round through the Kalman filter and clustered devices based on the value of the previous round. In contrast to a study that assumes a non-IID environment with a specific distribution pattern [4], we randomly assign the number of data for each class to each device and set it to be similar to that in a real non-IID environment. Our main contribution to this paper is as follows:

- 1) We introduce a Kalman filter-based clustering federated learning method called K-FL to resolve the practical challenges in FL. The key innovation of the proposed algorithm is to exploit a) the local updates of devices, and b) the accuracy of evaluating the model of the device that requested clustering with the local dataset of other devices.
- 2) We present the first (to the best of our knowledge) clustering FL method that can train a model requiring fewer communication rounds under the premise that totally non-IID environment without any prior knowledge and an initial value set by the user.

The rest of this paper is structured as follows: In Section II, we present preliminaries that introduce the process of FL. Section III includes a system model that includes our proposed Kalman-filter based clustering federated learning method (K-FL). Section IV then presents the experimental performance and numerical results of K-FL. Section V concludes our paper.

II. PRELIMINARIES

In this section, we introduce the FL process from the perspectives of devices and servers. We consider a set $\mathcal{U}$ of U devices and one server that jointly implements the FL algorithm. We assume that there is no global model in the first round of communication and that the objective function is given as follows:

$$\min_{\mathbf{g}} \frac{1}{N} \sum_{i=1}^U \sum_{n=1}^{N_i} f(\mathbf{g}, \mathbf{x}_{i,n}, \mathbf{y}_{i,n}). \quad (1)$$

Let $N = \sum_{i=1}^U N_i$ denote the total number of training data samples, and let $\mathbf{g}$ denote the model parameters; let N_i consist of an input vector $\mathbf{x}_{in} \in \mathbb{R}^{N_{in} \times 1}$ and its corresponding output vector $\mathbf{y}_{in} \in \mathbb{R}^{N_{out} \times 1}$, where $f(\mathbf{g}, \mathbf{x}_{i,n}, \mathbf{y}_{i,n})$ is a local objective function.

In this study, we use the stochastic gradient descent method as the optimizer [15]. The local model update based on the stochastic gradient descent method can be described as follows.

$$\mathbf{o}_{\gamma, \epsilon}^i = \mathbf{o}_{\gamma, \epsilon-1}^i - \frac{\lambda}{N_i} \sum_{n=1}^{N_i} \nabla f(\mathbf{o}_{\gamma, \epsilon-1}^i, \mathbf{x}_{i,n}, \mathbf{y}_{i,n}). \quad (2)$$

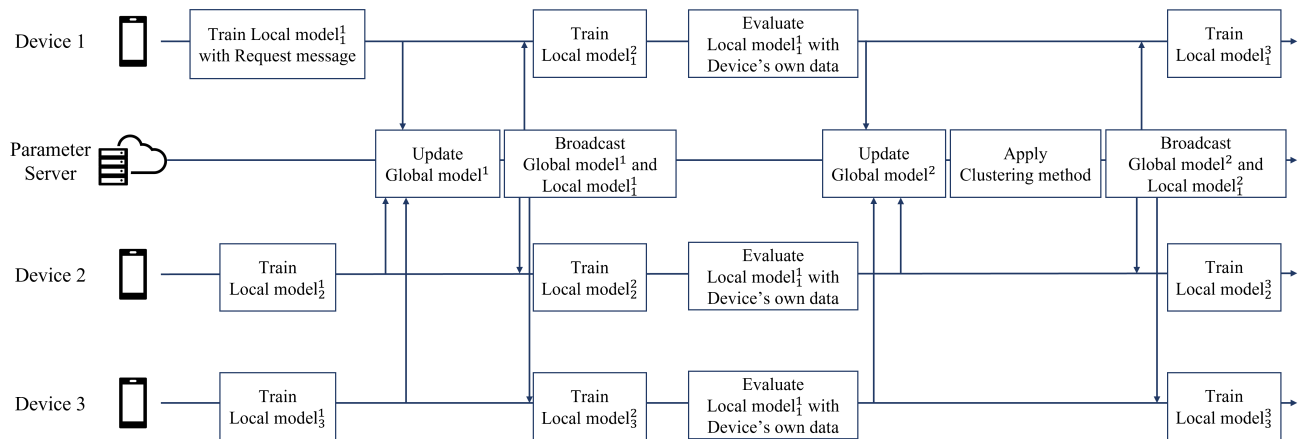


FIGURE 1. The flowchart expressing the progress of clustering.

Here, γ denotes the number of communication rounds, ϵ denotes the number of local iterations, $\mathbf{o}_{\gamma, \epsilon}^i$ denotes the local model when device i goes through communication round γ and local iteration ϵ , λ denotes the learning rate, and $\nabla f(\mathbf{o}_{\gamma, \epsilon-1}^i, \mathbf{x}_{i,n}, \mathbf{y}_{i,n})$ denotes the gradient of $f(\mathbf{o}_{\gamma, \epsilon-1}^i, \mathbf{x}_{i,n}, \mathbf{y}_{i,n})$.

After training for the local iteration determined for each device, the local model is transmitted to the server. When the local models of all the devices are sent to the server, the server updates the global model $\mathbf{g}$ as follows:

$$\mathbf{g}_{\gamma+1} = \frac{1}{N} \sum_{i=1}^U N_i(\mathbf{o}_{\gamma+1, \epsilon}^i - \mathbf{g}_{\gamma}) + \mathbf{g}_{\gamma}. \quad (3)$$

Here, $(\mathbf{o}_{\gamma+1,\epsilon}^i - \mathbf{g}_\gamma)$ denotes the local model parameter of device i trained through ϵ local iterations. We define one communication round as a round performing these steps once, and we repeat them for a set number of communication rounds.

III. PROPOSED K-FL

In this section, we introduce the proposed clustering method based on a Kalman filter called K-FL. The objective of this study is to provide an optimized global model for devices that request clustering. By providing an optimized global model to a device that requests clustering, we attempt to present an approach with improved performance and low training time compared to existing methods. In addition, the server only provides an optimized global model suitable for the device requesting clustering; however, it does not prohibit other devices not belonging to the cluster from training. This implies that devices that do not belong to the cluster are not affected in terms of the prediction accuracy and cross-entropy loss. Note that we define a global model optimized for a specific device as a cluster model.

In this study, we used the prediction accuracy as an evaluation measure for clustering and created a matrix based on the accuracy, as presented in Table 1. Before introducing

TABLE 1. Example of a matrix based on evaluation accuracy.

Device	Communication round				
	1	2	3	4	5
1	0.000	0.037	0.230	0.060	0.089
2	-0.016	-0.018	-0.035	-0.043	-0.040
3	-0.035	-0.054	-0.056	-0.070	-0.081
4	-0.016	-0.010	-0.028	-0.020	-0.018

the proposed K-FL, it is necessary to set the time at which clustering is to be performed, and for this, we used the Kalman filter. In the following section, we introduce the Kalman filter used in the proposed method and describe the clustering mechanism for the devices based on the aforementioned matrix.

A. SETTING OF A CLUSTERING POINT IN TIME USING A KALMAN FILTER

The setting of a clustering point in time usually occurs after a specific mobile device requests clustering. In Fig. 1, Device 1 denotes a device requesting clustering. As illustrated in Fig. 1, when a specific device (e.g., Device 1) requests clustering, the server broadcasts the local model of the device that requested clustering while broadcasting the global model to other devices. In this study, we used the prediction accuracy as an index for clustering and performed clustering based on the accuracy derived by evaluating the transmitted local model with the local data of each device.

However, if we perform clustering based on a single evaluation, the clustered result may not be particularly reliable; therefore, a standard for the number of clustering rounds must be set. Hence, in this study, we used the Kalman filter to set this standard.

Note that a Kalman filter is derived according to Bayes' theorem and is an algorithm that repeatedly performs

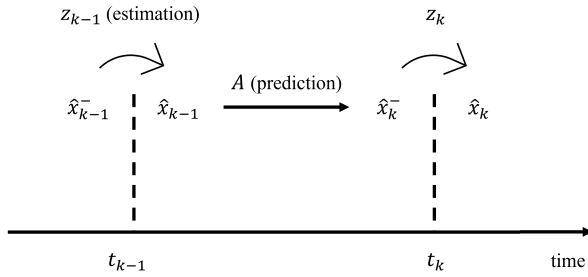


FIGURE 2. Process of prediction and estimation.

prediction and estimation for state estimation. Therefore, Bayes' theorem should be used to understand a Kalman filter. Notably, Bayes' theorem depicts the relationship between the prior probability $P(A)$ and the posterior probability $P(B|A)$ as follows:

$$P(B|A) = \frac{P(A|B) \cdot P(B)}{P(A)}. \quad (4)$$

The Kalman filter involves the process of inferring the probability $P(B|A)$ of an event based on previous experience $P(B)$ and present evidence $P(A)$. Thereafter, $P(A|B)$, which represents the likelihood of a hypothesis based on observed results, guarantees the reliability of the posterior probability. Therefore, Bayes' theorem presents the advantage of being able to derive reliable results with limited prior information.

Algorithm 1 Algorithm Presenting the Process Flow of a Kalman Filter

Input: z_k
Output: $\hat{x}_k$
Step 1. Set the initial value:
 $\hat{x}_0, P_0$
Step 2. Set the system model:
 A, H, Q, R
Step 3. Predict the estimation value and error covariance:
 $\hat{x}_k^- = A \cdot \hat{x}_{k-1}$
 $P_k^- = A \cdot P_{k-1} \cdot A^T + Q$
Step 4. Calculate the Kalman gain:
 $K_k = P_k^- \cdot H^T \cdot (H \cdot P_k^- \cdot H^T + R)^{-1}$
Step 5. Calculate the estimation value:
 $\hat{x}_k = \hat{x}_k^- + K_k(z_k - H \cdot \hat{x}_k^-)$
Step 6. Calculate the error covariance:
 $P_k = P_k^- - K_k \cdot H \cdot P_k^-$

The Kalman filter can predict the posterior data according to existing data. As depicted in Fig. 2, the input measurement value z_k is processed internally, and the estimated value $\hat{x}_k$ is output.

$$x_t = A \cdot x_{t-1} + w_{t-1}, \quad w_t \sim N(0, Q). \quad (5)$$

$$z_t = H \cdot x_t + v_t, \quad v_t \sim N(0, R). \quad (6)$$

(5) explains the manner in which the present state t is affected by the past state $t - 1$. This process, however, includes noise,

which affects the state variable w_{t-1} . (6) explains the manner in which the state affects the measurements. This process includes measurement noise defined as v_t . Note that a Kalman filter is derived according to Bayes' theorem and is an algorithm that repeatedly performs prediction and estimation for state estimation, as illustrated in Fig. 2, and continues to reflect the error between the predicted and measured values when estimating the state variables.

Algorithm 1 presents the prediction and estimation processes based on the Kalman filter. Here, we set the initial values $\hat{x}_0$ and P_0 and the system models A, H, Q , and R . Notably, the initial value of the system model including noise depends on the experience of the Kalman filter designer [16]. $x_k \sim N(\hat{x}_k, P_k)$ denotes the relationship between the error covariance P_k and the estimation value of the Kalman filter $\hat{x}_k$. **Step 3** denotes the prediction process, and we calculate the prediction values $\hat{x}_k^-$ and P_k^- according to the initial value and the system model. **Step 4-6** denote the estimation processes. **Step 4** calculates the Kalman gain K_k , where K_k acts as a weight for the Kalman filter, and **Step 5** calculates the estimated value $\hat{x}_k$ of a state variable using the input measurement value z_k . Finally, **Step 6** determines the estimation accuracy by calculating the error covariance P_k .

In this paper, we propose the use of a Kalman filter, a statistical theorem, to effectively set the point at which clustering is to be performed. As mentioned above, the Kalman filter relies on the designers' knowledge and experience of the system. Therefore, we set the values of the system model as $A = 1, H = 1, Q = 0.0179$, and $R = 0.0003$ based on the results of previous simulations [17]. w_k in (5) denotes the increase in accuracy for each communication round of the previous simulation, which is referred to as the system model setting, and v_k in (6) represents the increase in accuracy of the simulation considered in this study (i.e., observation value). We set the initial values as $\hat{x}_2 = 0.1703$ and $P_2 = 0.0013$ based on the value obtained when proceeding from communication rounds 1 to 2, and we set the point at which clustering is to be performed by applying the Kalman filter. Details pertaining to the application of the Kalman filter can be found in Appendix.

B. PROPOSED CLUSTERING METHOD

In this study, as stated, we use the prediction accuracy as an index to classify devices at a point set in the previous session. The matrix, including the accuracy of each device for each round, is presented in Table 1. The first row of the matrix indicates the increase/decrease in the accuracy of a device that requested clustering. Here, the accuracy of each device is a value obtained when the local model of the device that requested clustering is evaluated using the local dataset of each device. In addition, the difference between the accuracy of each device and the accuracy of the device that requested clustering is indicated in the second row.

After training in the first communication round, the first column of Table 1 is created. Thereafter, as each

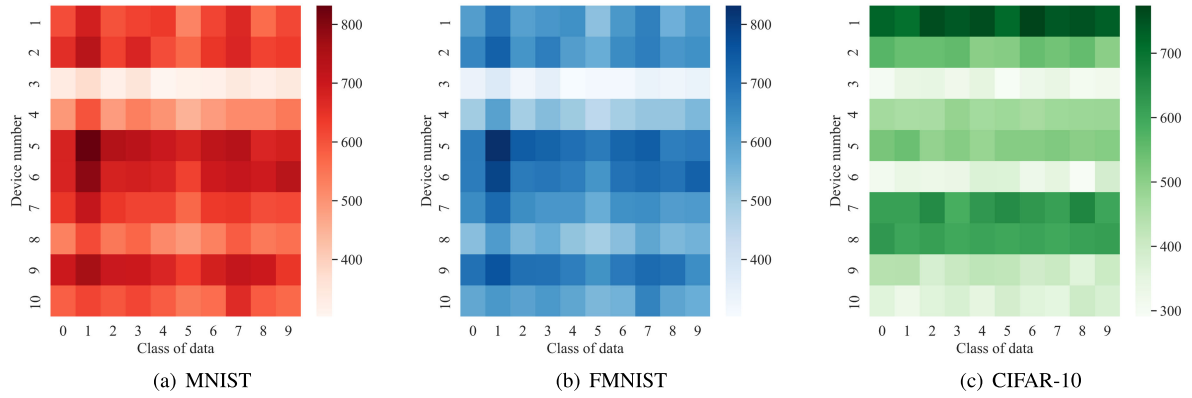


FIGURE 3. Data distribution of each dataset.

communication round proceeds, the columns are sequentially generated, and devices are clustered according to the accuracy of each device at the point at which clustering is to be performed. The expression describing the proposed clustering method is as follows.

$$\theta_{\tau}^i < \theta_{\tau-1}^i, \quad \forall i \in U, \quad \forall \tau \in \gamma. \quad (7)$$

θ_{τ}^i denotes the difference in accuracies between the i^{th} device and the device requesting clustering in the previous τ^{th} communication round, and $\theta_{\tau-1}^i$ denotes the difference in this accuracy in the $(\tau - 1)^{th}$ communication round. Based on (7), devices that continuously present a greater difference than the difference in the accuracies of the previous round are set to not be clustered. This implies that when comparing the accuracy of a device with that of the device requesting clustering, the device whose accuracy difference increases as the communication round progresses is excluded from the cluster.

After clustering, the cluster model and global model, wherein all devices participate, are operated independently, and depending on the operation time for each communication round of each model, the communication round in progress for each model at the same time may differ.

IV. SIMULATION RESULTS

In this section, we present the simulation results of the proposed method. For our analysis, we assume a situation wherein we do not have prior knowledge of the data distribution for each device. Notably, in FL-based studies, in the absence of prior knowledge on local datasets, the training of the global model is often challenging [18]. In the simulation conducted in this study, we aim to demonstrate improved accuracy by performing clustering with no prior knowledge of local datasets.

A. SIMULATION PARAMETERS

The simulation results present a comparison between the performance of a model to which clustering is applied and that of the global model in a FedAvg-based environment.

We use LeNet-5 [19] as the training model and the MNIST,¹ FMNIST,² and CIFAR-10³ datasets as the simulation datasets. We set the number of devices participating in FL to 10 and the number of communication rounds to 50. Thereafter, we allocate the number of classes of each device differently and build a simulation environment such that the data distribution is non-IID. The data distributions of each dataset allocated to each device are presented in Fig. 3. The cluster model illustrated in the simulation results is a model wherein only certain devices participate after clustering, whereas the global model is a model wherein all devices participate in FL. Through simulations, we aim to demonstrate that the cluster model presents improved performance compared with the existing global model. In addition, based on the performance of the global model in the same environment without the application of the proposed method, we aim to demonstrate that even if the proposed method were applied, the global model would present almost no loss in accuracy.

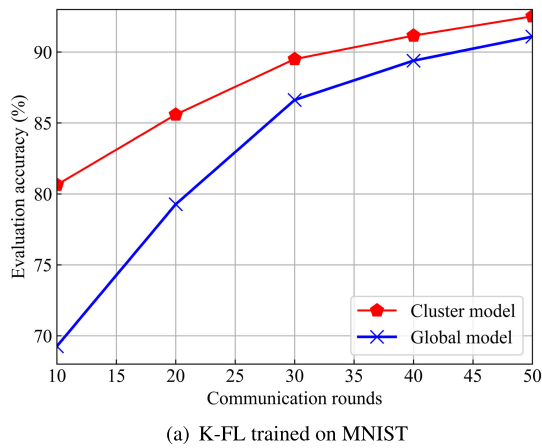
B. EVALUATION OF LEARNING PERFORMANCE

Fig. 4(a) presents the prediction accuracy of the global model and that of the cluster model on the MNIST dataset. In this figure, we can note the presence of a gap between the accuracies of the global and cluster models when a more suitable model is allocated to the device that requested clustering. As a numerical result, in the 50th communication round, the global model presents an accuracy of 91.09%, and the cluster model presents an accuracy of 92.52%, confirming an improvement in performance by 1.57% compared to the existing method. Although the performance of the cluster model appears to have improved based on the proposed method compared with that for the existing method, the numerical result is not particularly better because the data distribution is completely non-IID. In addition, the use of

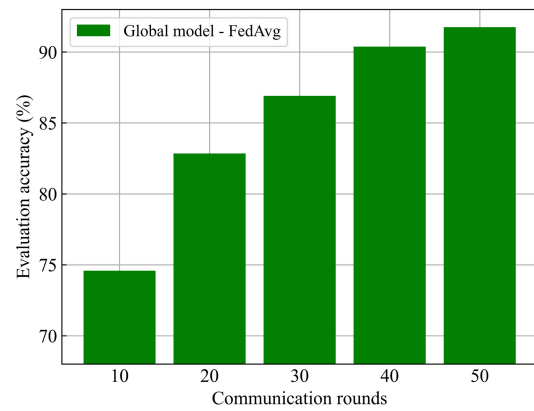
¹ Available at “<http://yann.lecun.com/exdb/mnist>”

² Available at “<https://github.com/zalandoresearch/fashion-mnist>”

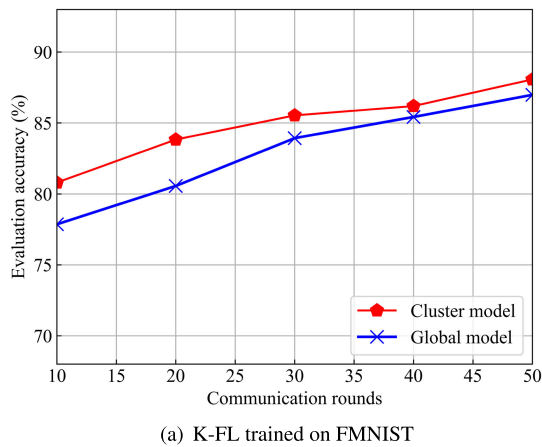
³ Available at “<https://www.cs.toronto.edu/~kriz/cifar.html>”



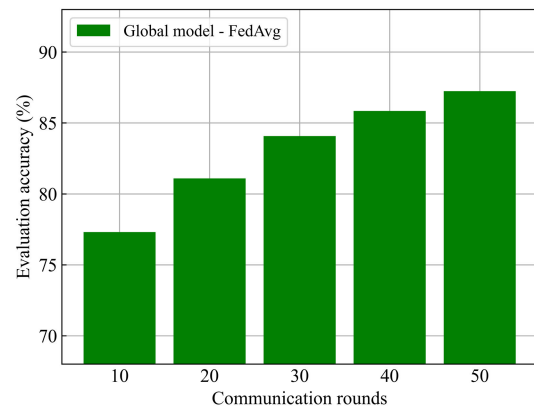
(a) K-FL trained on MNIST



(b) FedAvg trained on MNIST

FIGURE 4. Evaluation accuracy achieved on MNIST.

(a) K-FL trained on FMNIST



(b) FedAvg trained on FMNIST

FIGURE 5. Evaluation accuracy achieved on FMNIST.

LeNet-5, a lightweight convolutional neural network model, may be the reason for this poor performance.

Fig. 5(a) presents the prediction accuracy of the global model and that of the cluster model on the FMNIST dataset. Fig. 5(a) also indicates that the accuracy of the proposed cluster model is improved compared to that of the existing method. As a numerical result, in the 50th communication round, the global model presents an accuracy of 86.98%, and the cluster model presents an accuracy of 88.07%, confirming that the performance is improved by 1.25% compared to the existing method. The reason for this performance improvement not being particularly better is the same as that when testing with the MNIST dataset.

Fig. 6(a) presents the simulation results in terms of accuracy using CIFAR-10 dataset. The red line means the cluster model which K-FL is applied, and blue line means the global model which K-FL is applied. The trend of two lines is similar, but it can be seen that the cluster model consistently shows high performance of about 3%. In the 50th communication round, the global model presents an

accuracy of 73.58%, and the cluster presents an accuracy of 76.69%, confirming that there is almost a 3.11% performance difference in terms of accuracy between the two models. This is also a larger difference compared to the simulation results of MNIST and FMNIST. This is because the advantage which is gained when clustering similar devices in a relatively complex CIFAR-10 data distribution increase.

Fig. 4(b), Fig. 5(b), and Fig. 6(b) present the results obtained when the global model is trained with FedAvg without the application of the proposed method. The results indicate that the resulting accuracy is 91.76% when training with the MNIST dataset, 87.24% when training with the FMNIST dataset, and 73.71% when training with the CIFAR-10 dataset. Thus, it can be confirmed that the global model without the application of the proposed method demonstrates a slightly higher accuracy than the global model with the application of proposed K-FL. This can be attributed to the following: In K-FL, because the devices that train the local model based on the cluster model form a part of the global model, the global model produces certain results that are

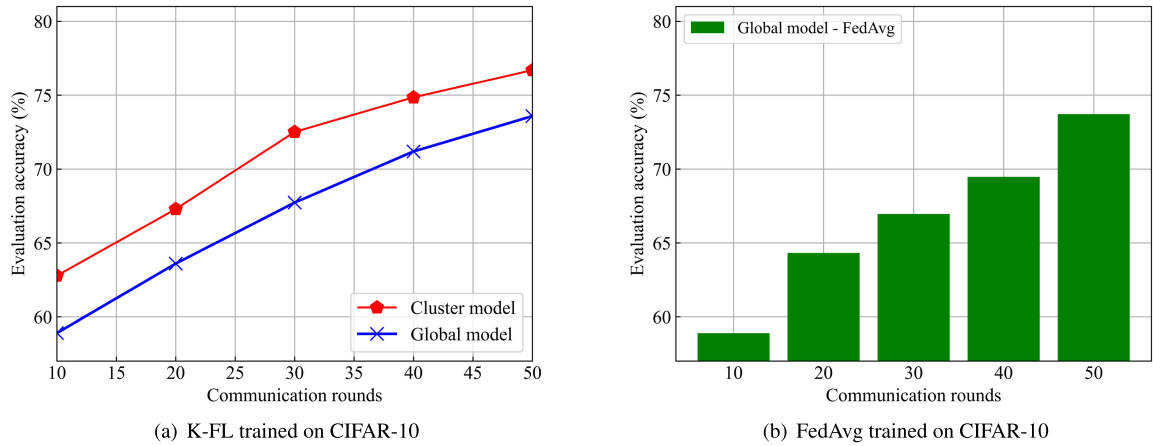


FIGURE 6. Evaluation accuracy achieved on CIFAR-10.

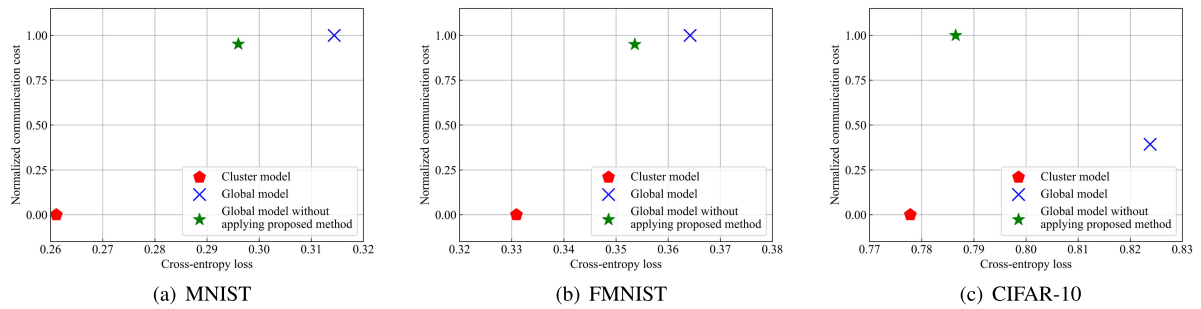


FIGURE 7. Comparison between K-FL and baselines in inference cross-entropy loss and communication cost space.

biased toward the cluster model. However, these results occur when reducing the operation time. To avoid even slight biases, the device participating in the cluster model can be made to create a local model based on the global model and can transmit it to the global model.

In Fig. 7, we measure the running time of communication including both transmitting and training the neural network and normalize it as the ratio to the communication time as reported communication cost. Fig. 7 shows that the proposed K-FL can improve the model while reducing the communication cost by comparing the cross-entropy loss and the normalized communication cost, rather than simply comparing the cross-entropy loss of both K-FL and FedAvg. In all three cases, the cluster model when K-FL is applied shows the lowest loss value and at the same time has the lowest normalized communication cost. There are two kinds of global models in Fig. 7. The blue line means the global model when K-FL is applied, and the green line means the global model in FedAvg. It is shown in Fig. 7 that that the application of K-FL does not negatively affect the communication cost and cross-entropy loss of the global model. Summarizing this section, in a situation that there is a device requesting clustering, it is shown that it is possible to achieve high performance with low communication cost by applying K-FL to provide a model suitable for the device.

TABLE 2. Computational complexity in different cases with different data and model.

Dataset	Method	Computational time cost (hour)
MNIST	K-FL	2.58
	FedAvg [4]	3.70
	LG-FedAvg [20]	2.63
FMNIST	K-FL	11.19
	FedAvg [4]	18.69
	LG-FedAvg [20]	13.27
CIFAR-10	K-FL	54.21
	FedAvg [4]	88.37
	LG-FedAvg [20]	62.74

C. COMPUTATIONAL COMPLEXITY

To obtain the computational complexity expressed as time complexity, it is assumed that all devices are in the same communication environment and use the same neural network. When the time complexity of Device 1 is defined as $\mathcal{O}(D_1)$, that of Device 2 is defined as $\mathcal{O}(D_2)$, and that of Device N as $\mathcal{O}(D_N)$, $\mathcal{O}(2)$ is the time complexity when two random devices participate in FL. In the real world, $\mathcal{O}(D_1)$, $\mathcal{O}(D_2)$, and $\mathcal{O}(D_N)$ may vary depending on the data

of each device, but since each device uses the same neural network (i.e., the number of trainable parameters is the same), regardless of which device, when two devices participate. The time complexity of is defined as $\mathcal{O}(2)$. That is, $\mathcal{O}(2) = \mathcal{O}(D_1) + \mathcal{O}(D_2) = \mathcal{O}(D_1) + \mathcal{O}(D_N)$ in our simulation. Using this, it can be shown that when K-FL is applied, it has improved time complexity compared to FedAvg. When K-FL is applied, cluster is formed with devices having similar data distribution, and at this time, the maximum number of devices participating to the cluster is the number of devices participating in the global model. Therefore, according to the definition of time complexity $\mathcal{O}(n)$, the time complexity of K-FL is equal to or lower than that of FedAvg in all situations.

Since each device has the same number of trainable parameters in the simulation, the time complexity varies according to the number of devices participating in global model training as shown in Table 2. Using the property that the number of devices participating in the cluster model of K-FL is always equal to or less than the number of devices participating in the global model, the time complexity of the cluster model is lower than that of FedAvg as shown in Table 2. LG-FedAvg, introduced in Table 2, is proposed with the goal of improving communication efficiency [20]. It is the state-of-the-art method that improves personalization and communication efficiency by jointly learning a global model and a compact local model of each device. Though LG-FedAvg also shows improved computational time cost compared to FedAvg in all datasets used in our simulation, K-FL shows improved performance than LG-FedAvg in all datasets in terms of computational time cost.

V. CONCLUSION

In this paper, we propose a clustering method called K-FL with the objective of overcoming the non-IID problem of the data of distributed devices participating in FL. For this, we assume a scenario wherein a device requested clustering and proposed a clustering method for a situation with no prior knowledge on the data distribution of devices. Generally, the decision pertaining to the execution of clustering should be prioritized, and in this study, we attempt to determine the point at which clustering is to be performed using the Kalman filter. Thereafter, based on the prediction accuracy of each device in each communication round, devices not belonging to the cluster are identified. The simulation results reveal that the proposed method demonstrates significantly improved performance in terms of the accuracy compared to the existing global model.

Our works have presented the role of the server (i.e., proposed Kalman filter-based clustering method) to provide an optimal model to the device, but considering the role of the device is also an interesting direction.

APPENDIX

First, we calculate the variance and standard deviation based on previous simulation results [17]. Table 3 presents the results of the 1st to 10th communication rounds, as reported

TABLE 3. Simulation results of a previous simulation.

Communication round	Prediction accuracy of the global model (%)
1	4.338884
2	9.586678
3	18.629338
4	23.542096
5	27.098122
6	31.480131
7	35.503882
8	39.142838
9	42.950973
10	47.296491

in [17]. Based on Table 3, we can obtain the variance and standard deviation as $V = 0.0179$ and $\sigma = 0.1340$, respectively. Hence, we know $\hat{x}_2 = 0.1703$, and $P_2^- = 0.0013$. Considering **Step 1** and **Step 2** in Algorithm 1, we can calculate $\hat{x}_3^-$ and P_3^- as follows:

$$\hat{x}_3^- = A \cdot \hat{x}_2 = 0.1703, \quad (8)$$

$$P_3^- = A \cdot P_2 \cdot A^T + Q_2 = 0.0013 + 0.0179 = 0.0192. \quad (9)$$

After we obtain $\hat{x}_3^-$ and P_3^- , we can calculate the Kalman gain K_3 as

$$\begin{aligned} K_3 &= P_3^- \cdot H^T \cdot (H \cdot P_3^- \cdot H^T + R) \\ &= 0.0192 \cdot (0.0192 + 0.0003)^{-1} = 0.9846. \end{aligned} \quad (10)$$

Based on (10), we have the following:

$$\begin{aligned} \hat{x}_3 &= \hat{x}_3^- + K_3(z_3 - H \cdot \hat{x}_3^-) \\ &= 0.1703 + 0.9846(0.2300 - 0.1703) = 0.2290. \end{aligned} \quad (11)$$

This completes the proof. $\square$

REFERENCES

- [1] E. Brynjolfsson and A. McAfee, "Artificial intelligence, for real," *Harvard Bus. Rev.*, vol. 1, pp. 1–31, Jul. 2017.
- [2] J. H. Ziegeldorf, O. G. Morchon, and K. Wehrle, "Privacy in the Internet of Things: Threats and challenges," *Secur. Commun. Netw.*, vol. 7, no. 12, p. 2728–2742, 2014.
- [3] M. Kim and J. Lee, "Information-theoretic privacy in federated submodel learning," *ICT Exp.*, vol. 2022, pp. 1–5, Feb. 2022.
- [4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.* PMLR, 2017, pp. 1273–1282.
- [5] W. Lee, "Reward-based participant selection for improving federated reinforcement learning," *ICT Exp.*, vol. 2022, pp. 1–6, Aug. 2022.
- [6] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, nos. 1–2, pp. 1–210, Jun. 2021.
- [7] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, vol. 2, 2020, pp. 429–450.
- [8] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 33, Dec. 2020, pp. 7611–7623.
- [9] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," 2019, *arXiv:1910.06378*.

- [10] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," 2020, *arXiv:2002.06440*.
- [11] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv:1806.00582*.
- [12] H. Zhu, J. Xu, S. Liu, and Y. Jin, "Federated learning on non-IID data: A survey," *Neurocomputing*, vol. 465, pp. 371–390, Sep. 2021.
- [13] C. Briggs, Z. Fan, and P. Andras, "Federated learning with hierarchical clustering of local updates to improve training on non-IID data," in *Proc. Int. Joint Conf. Neural Netw.*, Jul. 2020, pp. 1–9.
- [14] F. Sattler, K.-R. Müller, and W. Samek, "Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 8, pp. 3710–3722, Aug. 2021.
- [15] S. U. Stich, "Local SGD converges fast and communicates little," 2018, *arXiv:1805.09767*.
- [16] P. Kim and L. Huh, *Kalman Filter for Beginners: With MATLAB Examples*. Scotts Valley, CA, USA: CreateSpace, 2011, vol. 4.
- [17] H. Kim, Y. Kim, and H. Park, "Reducing model cost based on the weights of each layer for federated learning clustering," in *Proc. 12th Int. Conf. Ubiquitous Future Netw. (ICUFN)*, Aug. 2021, pp. 405–408.
- [18] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-IID data silos: An experimental study," in *Proc. IEEE 38th Int. Conf. Data Eng. (ICDE)*, May 2022, pp. 965–978.
- [19] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [20] P. P. Liang, T. Liu, L. Ziyin, N. B. Allen, R. P. Auerbach, D. Brent, R. Salakhutdinov, and L.-P. Morency, "Think locally, act globally: Federated learning with local and global representations," 2020, *arXiv:2001.01523*.



HYUNGBIN KIM was born in Seoul, South Korea. He received the B.S. and M.S. degrees in information and communication engineering from Myongji University, Yongin, South Korea. He is currently pursuing the Ph.D. degree with the Department of Computer Science and Engineering, Korea University, Seoul. His research interests include big data analysis, neural networks, deep learning, and mobile edge/cloud computing.



BYUNGCHAN KIM was born in Seoul, South Korea. He received the B.S. degree in information and communication engineering from Myongji University, Yongin, South Korea. He is currently pursuing the M.S. degree with the School of Information and Communication Engineering. His research interests include wireless networks, EDCA, mobile communication, and random access.



YONGHO KIM was born in Seoul, South Korea. He received the B.S. degree in computer software from Korean Bible University, Seoul. He is currently pursuing the integrated M.S. and Ph.D. degrees with the School of Information and Communication Engineering. His research interests include AI/ML modeling and prediction, intelligent system development, and computer vision.



CHEOLWOO YOU (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronics engineering from Yonsei University, Seoul, South Korea, in 1993, 1995, and 1999, respectively. From January 1999 to April 2003, he was a Senior Research Engineer with LG Electronics, Gyeonggi, South Korea. From 2003 to 2004, he was a Senior Research Engineer with EoNex, Songnam, South Korea. From August 2004 to July 2006, he was with Samsung Electronics, Suwon, South Korea. Since September 2006, he has been with the Department of Information and Communications Engineering, Myongji University, Gyeonggi. His research interests include next generation communication systems, artificial intelligence, air I/F technologies in the international standards, communication theory, signal processing, 5G/6G communication systems, machine and deep learning, AR/VR, the IoE/AIoT, and V2X.



HYUNHEE PARK (Member, IEEE) received the Ph.D. degree in electronics and computer engineering from Korea University, Seoul, South Korea, in 2011. She joined as a Faculty Member with Myongji University, in 2020, where she is currently an Associate Professor with the Department of Information and Communication Engineering. She is also a Supervisor with the Data Analysis and Networking Laboratory (DAN Laboratory). From March 2017 to February 2020, she was with the Department of Computer Software, Korean Bible University, Seoul, as an Assistant Professor. Since November 2014, she has been with LG Electronics as a Senior Researcher for Wi-Fi standardization (IEEE 802.11ax, Wake Up Radio, and Wi-Fi Alliance). From January 2013 to October 2014, she joined the INRIA Research Center as a Postdoctoral Researcher, where she works with the DIONYSOS Research Group and Telecom Bretagne as a Postdoctoral Researcher. She undertakes the system implementation for QoE on wireless networks with Telecom Bretagne. From September 2011 to February 2013, she joined Korea University as a Research Professor. Her research interests include wireless networks, mobile edge/cloud computing, and big data analysis. She served as the Program Co-Chair for the IMIS 2020, the Organizing Committee for the ICTC2020, ICTC2021, and BWCCA 2021, and a Workshop Organizer for FINGNet 2019 and 2020. She is a Guest Editor of the *Electronics* and *Journal of Advanced Transportation*.

...